DIO: Hybrid Cost Routing, Session Affinity, and Sound Admission for Multi-Instance LLM Serving over Stock vLLM

Keyush Nisar[1*], Krishil Parikh[1] and Krisha Maisheri[1]

[1*]Department of Computer Engineering, Dwarkadas J. Sanghvi College of Engineering, Mumbai, India.

*Corresponding author(s). E-mail(s): Nisarkeyush3@gmail.com; Contributing authors: Krishilparikh75@gmail.com; KrishaMaisheri16@gmail.com;

Abstract

If you run two or three stock vLLM servers and put a Round-Robin balancer in front, the balancer still treats every peer as equal. In practice they are not: one can sit on a deeper queue, hold a fuller KV cache, or simply run slower for a while—even when the GPUs are the same model. A learned latency model does not fully fix that. On dual Tesla T4 hardware our dual-timescale NLMS predictor has MAPE around 90–130%, which is usable for ranking but unsafe if you reject traffic because $\hat{y}$ alone exceeds an SLO.

We built **DIO** (`dio-serve`) as a thin OpenAI-compatible gateway that sits in front of stock engines and does three concrete things. First, it ranks backends with online NLMS, then nudges those scores with real engine state read from each vLLM Prometheus `/metrics` endpoint (KV usage, waiting requests, prefix-cache hits)—HTTP only, no engine patches. Second, it keeps multi-turn and agent sessions sticky to the replica that already holds the shared prefix, and records affinity hit rate. Third, it keeps admission off the unreliable absolute prediction: default modes are empirical (observed latency percentiles) and rank-only; absolute-$\hat{y}$ gating is left as a diagnostic that we show can starve healthy traffic.

On dual-T4 + Qwen2.5-3B + stock vLLM: (i) homogeneous NLMS is no better than Round-Robin on short probes ($\boldsymbol{n}$**=10**); (ii) under $\times$**2** service-time skew, NLMS cuts p99 by **48.3% ± 0.7%** vs. RR; (iii) on **real multi-turn** workloads ($\boldsymbol{n}$**=10**), hybrid metrics cut mean p99 by **40.9% ± 30.5%** vs. RR (win on **8/10** seeds; median p99 ∼51% lower) and beat hybrid-off; affinity stickiness is **1.00** vs. **0.50** for RR (**0.998** vs. **0.75** under concurrent load) with hit rate ∼0.91–0.93; absolute admission rejects **39.5% ± 21.0%** under a tight SLO while empirical/rank_only reject **0%**. Source code is public. Multi-SKU fleets are out of scope.

Keywords: LLM serving, multi-instance load balancing, vLLM, hybrid routing, prefix-cache affinity, admission control, NLMS, Prometheus metrics.

1 Introduction

1.1 Motivation

Practitioners commonly run *multiple* stock vLLM [1] (or SGLang/TGI [2–4]) instances and put Nginx [5], Kubernetes Services [6], Envoy [7], or Ray Serve [8] in front with Round-Robin or least-connections. Single-node engines optimize continuous batching [9, 10]; they do not solve *which backend* should take the next request. Classic L7 balancers [5, 11] treat backends as equal capacity units.

In production, effective cost still diverges even on identical GPU SKUs: pending queues grow unevenly, KV-cache utilization spikes on one replica, prefix caches favor sessions pinned to a peer, and host interference can temporarily slow one process. The result is avoidable tail latency [12]. Co-designed cluster schedulers [13–15] can react with deep engine hooks or live KV migration, but many operators need a non-invasive front door that wraps stock OpenAI-compatible [16] engines without patches.

A second problem is quieter but equally important: online latency models learned from black-box e2e telemetry are often accurate only as *relative rankers*. On dual Tesla T4 we measure dual-timescale NLMS [17] MAPE of ∼90–130%. Systems that still gate admission on "reject if $\min \hat{y} > \text{SLO}$" inherit that error and can starve healthy traffic.

1.2 Approach

We design **DIO** (`dio-serve`) as a thin control plane for the multi-instance pattern (Figure 1). The closed loop uses coarse request telemetry (e2e latency, tokens, gateway pending) *and*, when available, stock engine metrics already exported on each vLLM Prometheus [18] endpoint—still HTTP-only, still zero source modification. Three mechanisms work together:

(A) **Hybrid cost routing.** NLMS provides an online ranking signal from completions; scraped KV usage, waiting requests, and prefix hit rate supply ground-truth corrections that close part of the MAPE/telemetry gap without requiring engine-specific hooks (Section 4).

(B) **Session/prefix affinity.** Multi-turn and agent clients [19, 20] benefit when follow-ups land where the shared prefix is already cached [21–23]. DIO scores that stickiness explicitly and reports affinity hit rate (Section 5).

(C) **Sound admission.** Ranking always uses $\arg\min S_w$; admission is a separate policy (`rank_only`, `empirical`, or diagnostic `absolute`) so a poorly scaled absolute $\hat{y}$ cannot thrash reject/admit decisions [24, 25] (Section 6).

1.3 Results preview

Dual-timescale NLMS does not invent wins when backends are equal: on homogeneous dual-T4 short probes, NLMS p99 is slightly worse than Round-Robin (Section 7.3). Under controlled $\times 2$ service-time skew, NLMS cuts p99 by 48.3%$\pm$0.7% vs. RR and beats d=2 RLS (Section 7.6). On **real dual-T4 multi-turn** runs (n=10), title mechanisms hold: hybrid scrape improves p99 vs. hybrid-off and RR on most seeds (Section 7.7); affinity stickiness dominates RR even under concurrent load (Section 7.8); absolute admission over-rejects under a tight SLO while empirical/rank_only do not (Section 7.9).

1.4 Contributions

1. **Hybrid cost routing (non-invasive).** Joint cost fusing online NLMS with scraped vLLM `/metrics`; real dual-T4 multi-turn gain vs. hybrid-off and RR (n=10).
2. **Session/prefix affinity for multi-turn routing.** Prefix-hash stickiness plus engine prefix-hit bonus; dual-T4 stickiness 1.00 vs. 0.50 RR (concurrent= 1) and 0.998 vs. 0.75 under concurrent load (n=10).
3. **Sound admission under unreliable $\hat{y}$.** Rank-only / empirical / absolute modes; on the real gateway with a tight SLO, absolute rejects $\sim$40% of requests while empirical and rank_only reject none (n=10).
4. **Open gateway and multi-seed evaluation.** Open-source `dio-serve`; dual-T4 + Qwen2.5-3B [26]; no free lunch on homogeneous short probes; multi-SKU fleets out of scope.

2 Background and related work

2.1 Single-node engines and prefix/KV systems

vLLM [1] popularized PagedAttention and continuous batching; alternatives revisit memory management without paging [27]. SGLang [2, 4] improves structured decoding and RadixAttention prefix reuse. TGI [3] targets production APIs. Hybrid iteration policies include Sarathi-Serve [10] and FastServe [28]. Disaggregated prefill/decode systems [25, 29–31] optimize pipeline stages and KV placement. Prefix/KV reuse [21–23, 32, 33] raises the value of *where* a request lands. These systems excel on one node or a co-designed cluster; multi-worker routing in front of *stock* engines remains external.

2.2 Cluster orchestrators and classical serving

Ray Serve [8] and Triton [34] emphasize actors and model repositories with coarse metrics. Earlier DNN serving stacks (Clipper [35], INFaaS [36], Clockwork [24]) established predictability and model-selection themes that LLM gateways inherit. Orca [9] advances iteration-level batching. AlpaServe [37] and CoCoServe [38] focus on placement and multiplexing. Mélange [39] targets heterogeneous capacity tables. NexusSched [13] combines predictive routing with engine co-design and multi-feature predictors ($O(d^2)$ updates). Llumnix [14] migrates live KV state. LoongServe [15] targets long-context elastic parallelism. ServerlessLLM [40] optimizes cold-start loading.

Metrics-aware gateways vs. DIO's hybrid scrape.

Production L7 balancers (Envoy [7], Nginx [5], Maglev [11]) and orchestrators such as Ray Serve can weight backends by *operator-defined* health or load metrics, and many stacks can scrape Prometheus in principle. What they typically do *not* do is fuse those engine-internal gauges with an online per-request latency ranker under an explicit admission policy that refuses to trust absolute $\hat{y}$. Co-designed LLM schedulers that use rich engine state usually require patches, shared memory, or KV migration [13, 14]. DIO's hybrid path is deliberately narrower and non-invasive: HTTP GET of each stock vLLM `/metrics` endpoint [1, 18], scalar cost terms in the same units as NLMS ranking, and graceful zero when scrape fails. We do not claim a bake-off against every Ray Serve metrics plugin; we claim a request-level gateway that combines dual-timescale NLMS, Prometheus engine state, multi-turn affinity, and decoupled admission without modifying engines.

2.3 Adaptive prediction, queues, and admission

NLMS stability and step-size practice follow adaptive filter theory [17, 41]; recursive least squares is the natural $O(d^2)$ comparator. Queue wait uses a batch-amortized Little-style heuristic [42, 43]. SJF-style ranking motivates predicted service routing [44]. Roofline [45] inspires soft memory ceilings recast as routing penalties. Goodput- and SLO-aware LLM serving [25, 46, 47] motivates treating admission as a separate policy from ranking. Simulation frameworks such as Vidur [48] characterize offline capacity; DIO targets online multi-instance ranking under weak absolute models.

3 System design

3.1 Architecture

Figure 1 is the system we ship and evaluate as `dio-serve`. Clients call the OpenAI HTTP API. The control plane (FastAPI + Python scheduler) ranks backends with NLMS, applies hybrid metrics, affinity, and admission, then proxies to a stock engine. On completion it updates per-worker models from e2e latency and token counts. A Go/gRPC research path exists in the repository but is not required for the results below.

Telemetry contract.

Four channels close the loop: (i) wall-clock e2e latency at the gateway; (ii) token counts from response `usage` or a tokenizer estimate; (iii) per-backend pending depth inside DIO; (iv) optional hybrid scrape of each backend `/metrics`. Soft/hard VRAM can come from config or from KV usage. We do not claim continuous NVML SM/power/temperature, and we do not patch engine source [13, 14].

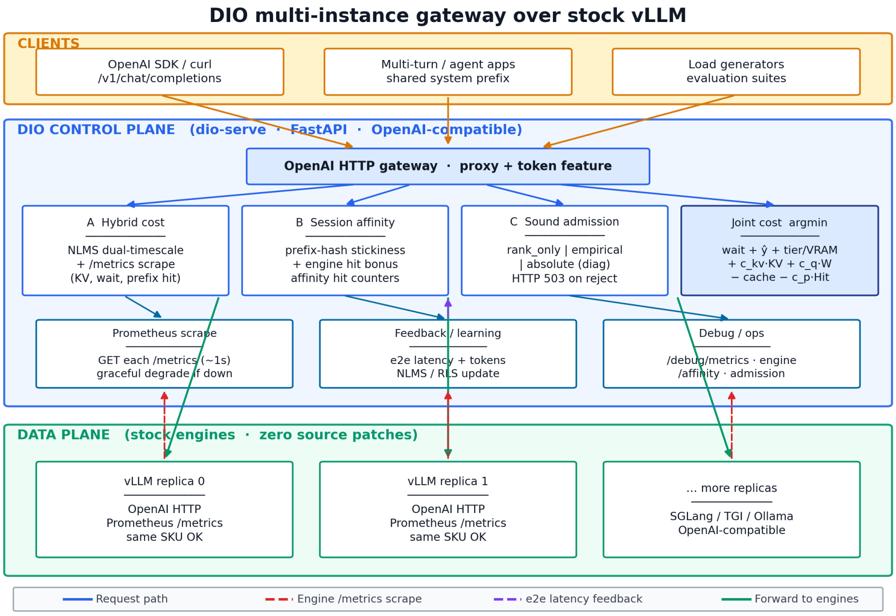


Fig. 1 DIO multi-instance gateway. OpenAI clients call `dio-serve`; the control plane combines NLMS ranking, Prometheus `/metrics` scrape, session affinity, and decoupled admission over stock vLLM replicas (no engine patches; no multi-SKU assumption).

3.2 Dual-timescale NLMS latency model

For worker w and approximate token count N, we model

$$\hat{y}_w = s_w N + b_w, \tag{1}$$

where s_w is ms/token and b_w is fixed overhead. On completion with residual $e = y - \hat{y}$, NLMS updates

$$s \leftarrow s + \mu \frac{e}{N}, \qquad b \leftarrow b + \mu_b e, \tag{2}$$

with dual rates $\mu_{\text{fast}}{=}0.1$, $\mu_{\text{slow}}{=}0.01$, $\mu_b{=}0.005$, and

$$s^{\text{eff}} = \alpha s^{\text{fast}} + (1-\alpha)s^{\text{slow}}, \quad \alpha{=}0.8. \tag{3}$$

Cold-start priors are $s_0{=}2.0\,\text{ms/token}$, $b_0{=}150\,\text{ms}$. We distinguish (i) $O(1)$ arithmetic per update from (ii) time-to-usable ranking: typically **15–20 completions per worker** before slopes leave priors. Figure 2 illustrates dual-timescale response to an injected slope jump in simulation. Token feature N prefers Hugging Face tokenizer counts plus planned `max_tokens`; fallback is $\lfloor|\text{prompt}|/4\rfloor +$ `max_tokens`.

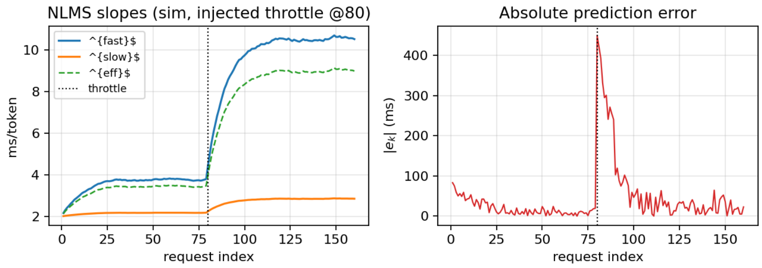


Fig. 2 NLMS dual-timescale adaptation under an injected true-slope jump at request 80 (simulation). s^{fast} reacts first; s^{slow} lags. Synthetic illustration, not dual-T4 wall-clock traces.

3.3 Baseline joint cost

Absent hybrid metrics, each healthy worker scores

$$S_w^{\text{base}} = \text{wait}_w + \hat{t}_w + \text{tierCost}_{r,w} + \text{vramCost}_w - \text{cacheBonus}_w, \tag{4}$$

with $\hat{t}_w = s_w^{\text{eff}} N + b_w$, $\text{wait}_w = (Q_w/B)\bar{t}_w$ (B=8), soft VRAM penalty when free VRAM <4 GB, hard exclusion when free VRAM <2.4 GB and N>1000, and tier mismatch soft cost 500 ms (hard block for large-on-small). Route to $\arg\min_w S_w$ in $O(|W|)$ time. Strategies also include Round-Robin, Least-Loaded, static slopes, and d=2 RLS as head-to-head baselines under the *same* cost shell when applicable.

3.4 Implementation notes

Launch with `dio serve -b e0=URL0 -b e1=URL1`. Background tasks scrape `/metrics` (default 1 s) and probe health. Debug endpoints: `/debug/metrics`, `/debug/engine`, `/debug/affinity`, `/debug/admission`. Pick overhead is about 10–15 μs (Section 7.2).

4 Hybrid cost routing with engine metrics

4.1 Problem: black-box ranking misses engine ground truth

NLMS observes only gateway e2e latency and tokens. That signal is delayed, noisy, and confounded by batching. Meanwhile stock vLLM already exports internal state via Prometheus [18]: GPU KV-cache utilization, number of waiting and running requests, and prefix-cache hit counters. Prior multi-instance balancers either ignore these signals or require invasive co-design [13, 14]. DIO's hybrid design is deliberately coarse: **scrape what the engine already publishes**, fold it into the same millisecond-scale cost used for ranking, and keep zero source modification.

4.2 Metric scrape and snapshot

For each backend i with base URL U_i and metrics path (default `/metrics`), a background task issues `GET` U_i`/metrics` and parses Prometheus exposition text. Metric name variants across vLLM versions are handled (e.g. `vllm:gpu_cache_usage_perc` vs.

`vllm:kv_cache_usage_perc`; prefix hit rate from a direct gauge or from hits/queries counters). Each scrape yields a snapshot

$$\mathcal{E}_w = (\mathrm{KV}_w,\, W_w,\, R_w,\, \mathrm{Hit}_w,\, \mathrm{ok}_w), \tag{5}$$

where $\mathrm{KV}_w \in [0,1]$, W_w and R_w are waiting/running request counts, $\mathrm{Hit}_w \in [0,1]$ is prefix-cache hit rate, and ok_w is false on HTTP/parse failure. On failure, hybrid terms contribute zero (graceful degrade to NLMS-only ranking). Scrape interval defaults to 1 s—fast enough to track KV pressure, slow enough not to contend with decode.

4.3 Hybrid joint cost

When ok_w is true, the full score is

$$\begin{aligned} S_w = {} & \mathrm{wait}_w + \hat{t}_w + \mathrm{tierCost}_{r,w} + \mathrm{vramCost}_w \\ & + c_{\mathrm{kv}}\,\mathrm{KV}_w + c_q\,W_w - \mathrm{cacheBonus}_w - c_p\,\mathrm{Hit}_w. \end{aligned} \tag{6}$$

Default hybrid coefficients (same ms units as other terms): c_{kv}=800, c_q=50 per waiting request, c_p=150. These were chosen so that a near-full KV cache (KV≈1) contributes on the order of one long-request decode budget, a moderate waiting queue (W≈10) is comparable to a soft VRAM penalty, and a high engine prefix hit rate is of similar scale to the session affinity bonus C=200 ms—heuristic parity with existing base cost terms. Base tier/cache/VRAM coefficients in prior L4 pilots showed ±50% relative insensitivity; for the hybrid triple we report a controlled library ±50% sensitivity check (Section 8) and note that a full dual-T4 grid search of $(c_{\mathrm{kv}}, c_q, c_p)$ remains open. Operators may override all three via `DIO_*` configuration. The dual-T4 multi-turn headline (Section 7.7) uses the defaults above.

Additionally, if KV_w>0.95 and N>500, worker w is hard-blocked for that request (engine-reported near-full KV). Free-VRAM estimates may be updated as $\mathrm{free}_w \approx \mathrm{total}_w(1 - \mathrm{KV}_w)$ when totals are known.

Scope of hybrid terms.

Hybrid cost is not a claim of SM utilization, power, or temperature awareness. It is also not a substitute for NLMS: when backends differ only in learned service slope and metrics look similar, NLMS ranking still dominates. The contribution is the combination of a lightweight online ranker with non-invasive engine ground truth, deployable as a stock gateway.

5 Session and prefix-cache affinity

5.1 Problem: multi-turn locality is underserved by RR

Round-Robin and least-connections scatter multi-turn sessions across replicas. Each turn that lands on a cold replica re-prefills shared system prompts and agent context, missing in-engine prefix caches [4, 21, 22]. Cache-aware and prompt-scheduling systems [23, 32] show large gains when locality is preserved, but they often assume

Algorithm 1 Worker selection with hybrid scrape, affinity, and admission

Require: request r, workers W, mode m, optional snapshots $\{\mathcal{E}_w\}$

Background (async): periodically scrape each backend `/metrics` into $\mathcal{E}_w$ (KV, W, Hit); on failure leave hybrid terms at 0
$N \leftarrow \text{TokenFeature}(r)$; $w^\star \leftarrow \perp$; $S_{\min} \leftarrow \infty$
for all $w \in W$ healthy and tier-feasible **do**
  $\hat{t}_w \leftarrow \text{Predict}(w, N)$
  if VRAM/KV hard-blocked$(w, N, \mathcal{E}_w)$ **then continue**
  end if
  $S_w \leftarrow \text{wait}+\hat{t}_w+\text{tier}+\text{vram}+c_{\text{kv}}\text{KV}_w + c_q W_w$ $-\text{cacheBonus}(r, w) - c_p\text{Hit}_w$
  if $S_w < S_{\min}$ **then** $S_{\min} \leftarrow S_w$; $w^\star \leftarrow w$
  end if
end for
if $w^\star = \perp$ **or** AdmissionReject$(S_{\min}, m)$ **then**
  return HTTP 503 with Retry-After
end if
RecordPrefixAffinity$(r, w^\star)$; **return** $w^\star$

co-designed clusters. DIO targets the common case: several stock replicas behind one OpenAI gateway, multi-turn or agentic clients [19, 20], no engine patches.

5.2 Affinity mechanism

On each admitted request with prompt text p, DIO computes a stable prefix hash over the first 256 characters:

$$h(p) = \text{hash}(p[0{:}256]) \bmod 2^{32}. \tag{7}$$

A map $M : h \mapsto w$ records the last worker that successfully served that prefix. When scoring worker w, if $M[h(p)] = w$, a cache affinity bonus $\text{cacheBonus}_w = C$ (default C=200 ms; raised in multi-turn experiments) is subtracted from S_w. Independently, the hybrid term $c_p\text{Hit}_w$ prefers workers the *engine* reports as currently caching well. After admission, DIO sets $M[h(p)] \leftarrow w^\star$.

Metrics.

DIO maintains affinity_decisions and affinity_hits (true when the chosen worker already owned the prefix). Exposed hit rate affinity_hits/affinity_decisions is the primary stickiness metric for multi-turn evaluation. When live vLLM metrics are available, operators can also compare engine-reported prefix hit rates under DIO vs. RR.

Relationship to in-engine caching.

Affinity is request-level stickiness complementary to RadixAttention [4]: DIO increases the chance that the engine's own cache is warm; it does not implement a second cache. The `no_cache` ablation disables the bonus for sensitivity studies.

6 Sound admission under unreliable latency prediction

6.1 Problem: absolute gates trust a weak model

Many predictive schedulers implicitly assume $\hat{y}$ is calibrated enough to compare against an SLO in absolute milliseconds. Section 7.4 shows that assumption fails for NLMS on dual-T4 (MAPE ∼90–130%). Cold-start priors and batching can make S_w larger than any reasonable SLO even when true latency is fine. Gating on $\min_w S_w >$ SLO then rejects healthy traffic—or, if priors are optimistic, admits work that will miss the SLO. Predictability-first serving [24] and goodput-oriented LLM systems [25, 46] motivate treating admission as a policy that must not inherit model bias.

6.2 Three admission modes

Ranking is always $\arg\min_w S_w$ among feasible workers. Admission is separate:

`rank_only`
: Never reject for SLO based on S_w. Only hard VRAM/tier/KV blocks apply. NLMS is used purely for ranking. Preferred for routing A/B tests and when the operator handles overload externally.

`empirical` (default)
: Maintain a rolling window of observed e2e latencies (default length 64). After warm-up (≥ 8 samples), compute the p-th percentile (default p=95). Reject only if that empirical tail exceeds the SLO *and* the best current score lies in the worst quartile of available worker scores. Cold-start never rejects on $\hat{y}$ alone.

`absolute` (legacy / diagnostic)
: Reject if $\min_w S_w >$ SLO. Sensitive to MAPE and cold-start scale. Retained so operators can quantify how often absolute would disagree with the active mode.

Algorithm 2 AdmissionReject($S_{\min}, m$)

Require: best score $S_{\min}$, mode m, SLO L, recent e2e window $\mathcal{Y}$
if $m =$ `rank_only` **then return** false
end if
if $m =$ `empirical` **then**
  if $|\mathcal{Y}| < 8$ **then return** false ▷ warm-up: do not trust $\hat{y}$
  end if
  $q \leftarrow$ Percentile($\mathcal{Y}, p$)
  $Q_{75} \leftarrow$ Percentile of current feasible scores at 75%
  return $(q > L) \wedge (S_{\min} \geq Q_{75})$
end if
return $(S_{\min} > L)$ ▷ `absolute`

Diagnostics.

Counters `would_reject_absolute`, `would_admit_absolute`, and `absolute_vs_active_disagree` record, on every pick, whether absolute mode would have made a different decision. These are the primary paper metrics for admission safety (Section 7.9). Rejected requests return HTTP 503 with `Retry-After`.

7 Experimental evaluation

7.1 Methodology

Hardware and software (real GPU).

Dual NVIDIA Tesla T4, stock vLLM [1], Qwen2.5-3B-Instruct [26], Hugging Face tokenizer, `dio-serve` gateway. Workshop suite: `run_workshop_final_suite.py`.

Regimes (non-interchangeable).

1. **Regime A — homogeneous dual-T4 (real):** identical peers, `max_tokens`=32, n=10 seeds × 30 requests.
2. **Regime B — legacy Locust pilot (secondary):** longer-decode ShareGPT [49]/arXiv/Azure-style traces, single seed; absolute seconds not mixed with Regime A ms.
3. **Regime C — service-time skew:** (i) CPU multi-seed simulation n=10; (ii) real dual-T4 with e1 behind delay-proxy ×2, n=10.
4. **A/B/C real multi-turn / admission (primary for title mechanisms):** dual-T4 + stock vLLM, multi-turn long system prefixes, live `/metrics` scrape, n=10 seeds (`run_gpu_abc_suite.py`), including affinity under concurrent load; library multi-seed suite remains a controlled complement (Sections 7.7–7.9).

Baselines and metrics.

Round-Robin (RR), Least-Loaded (LL), RLS [17] (d=2, λ=0.99) under the same engines where applicable. Primary metrics: p50/p99 e2e mean±std; MAPE; fraction to backend e0; affinity hit rate / stickiness; admission reject rate, completions, absolute-vs-active disagreement. **MAPE under RR/LL:** DIO still runs a *shadow* dual-timescale NLMS predictor on every completion for diagnostics even when the routing strategy is RR or LL; the MAPE column is that shadow model, not evidence that RR routes by $\hat{y}$. We do not claim a matched multi-node Orca [9] or vLLM-distributed bake-off.

7.2 Control-plane overhead

On a scalability microbench with many concurrent mock workers, instrumented scheduling completes in approximately **14 μs** per request (worker scan, cost evaluation, prefix hash). Per-worker state is ∼256 B. The control plane is not the bottleneck relative to LLM decode (Figure 3).

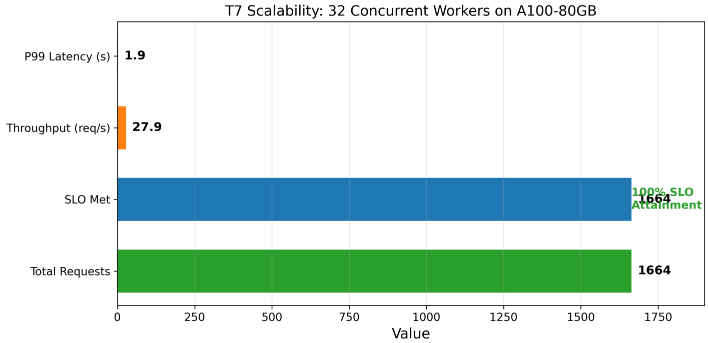


Fig. 3 Control-plane scalability microbench. Scheduling overhead remains ∼14 μs; plotted e2e latency is synthetic worker delay.

7.3 Regime A: homogeneous dual-T4 (no free lunch)

Table 1 is the primary homogeneous multi-seed matrix. On *identical* backends, **NLMS does not beat Round-Robin on mean p99** (slightly worse: $-2.6\% \pm 1.0\%$). This is the correct outcome if ranking is driven by learned service cost and workers truly look alike. Secondary signals: NLMS keeps tight p99 std (31 ms) while Least-Loaded is sticky (100% to e0, p99 std 430 ms); RLS mis-routes (frac e0 0.12).

Table 1 Regime A (real, primary). Dual-T4 + vLLM + Qwen2.5-3B, HF tokenizer, n=10 × 30, `max_tokens`=32. Mean±std.

Strategy	p99 (ms)	MAPE (%)	frac e0	vs RR
NLMS	3413 ± 31	123 ± 6	0.45 ± 0.04	$-2.6 \pm 1.0\%$
RLS (d=2)	3466 ± 11	132 ± 6	0.12 ± 0.02	$-4.2 \pm 0.6\%$
Round-Robin	3326 ± 17	88 ± 2	0.50 ± 0.00	—
Least-Loaded	3313 ± 430	141 ± 13	1.00 ± 0.00	$+0.4 \pm 13\%$

7.4 MAPE versus relative ranking

NLMS MAPE on dual-T4 is **high** ($123 \pm 6\%$ with tokenizer). HF tokenizer vs. byte heuristic (W2, n=3): MAPE 117.5±10.9% vs. 89.7±4.9%—tokenizer is **worse**, so error is structural (linear model, cold start, batching), not mere token counting. Routing uses $\arg\min S_w$; ranking under skew is the success criterion (Regime C). Absolute admission that trusts $\hat{y}$ is therefore unsafe (Section 7.9).

7.5 Regime B: legacy Locust pilot (secondary)

Table 2 retains a single-seed Locust pilot (longer decode, four logical workers). Absolute seconds are **not** comparable to Regime A milliseconds; we report it only for continuity with earlier pilots.

Table 2 Regime B (legacy single-seed Locust pilot). Not multi-seed; different setup from Table 1.

Trace	Strat.	p99 (s)	RPS	Fail %
ShareGPT	NLMS	**67**	**0.37**	0
ShareGPT	RR	81	0.29	0
arXiv	NLMS	52	**0.36**	0
arXiv	RR	51	0.30	0
Azure	NLMS	48	0.33	0
Azure	RR	47	0.36	0

7.6 Regime C: service-time skew

***Simulation** (n=10).*

With distinct true (b, s) pairs, NLMS places 97.5% of traffic on the fast worker and cuts p99 by 38.3% $\pm$ 2.0% vs. RR (Table 3). RLS fails to rebalance (frac fast 0.40 $\pm$ 0.18). Local `pick()` overhead: NLMS $\approx$ 9.6 μs, RLS $\approx$ 9.4 μs at d=2.

Table 3 Regime C (simulation). Slope-skew dual workers, n=10 $\times$ 200.

Metric	NLMS	RR	Notes
Frac. to fast	0.975 $\pm$ 0.000	0.500 $\pm$ 0.000	
p99 (sim units)	1411 $\pm$ 34	2287 $\pm$ 49	
p99 vs RR	38.3% $\pm$ 2.0%		

***Real dual-T4 service-time skew** (n=10).*

We inflate e1 wall-clock e2e by $\times$2 via `latency_delay_proxy` while still serving real tokens on a real T4—a controlled *slow peer*, not a second GPU SKU. This is an **artificial proxy** for the intro's organic divergence (thermal throttling, co-tenant noise, uneven queues); we measure the proxy reproducibly, not a natural interference trace. Table 4: NLMS p99 3427 $\pm$ 38 ms vs. RR 6632 $\pm$ 37 ms (48.3% $\pm$ 0.7% improvement). RLS is weaker and noisier (23.2% $\pm$ 22.9%). Mean frac to e0 under NLMS is modest (0.47 $\pm$ 0.08). NLMS p99 is close to Regime A (3413 $\pm$ 31 ms) because the router largely avoids the delayed peer on the same hardware and probe workload; RR is much worse (6632 ms) because half the traffic hits the $\times$2 peer. Homogeneous longer decode (`max_tokens`=128, n=3 $\times$ 20) stays tied with RR (13835 $\pm$ 73 vs. 13821 $\pm$ 88 ms).

7.7 Hybrid cost on real dual-T4 multi-turn

Subsections 7.7–7.9 are the **primary real-hardware** checks of the title mechanisms, using dual stock vLLM on Tesla T4, Qwen2.5-3B, HF tokenizer, and live Prometheus `/metrics` scrape (`run_gpu_abc_suite.py`, n=10 seeds). Workload: multi-turn sessions with long shared system prefixes (agent-style), 12 sessions $\times$ 4 turns, `max_tokens`=32, with light concurrency on G1 to build real queue/KV activity. Metrics probe confirmed scrape `ok` on both engines before load.

Table 4 Regime C (real GPU, primary). Dual-T4 + delay-proxy ×2, n=10 × 30, HF tokenizer. NLMS p99 near Table 1 is expected (avoids slow peer), not a copy artifact.

Strategy	p99 (ms)	frac e0	vs RR	MAPE (%)
NLMS	$\mathbf{3427 \pm 38}$	0.47 ± 0.08	$\mathbf{48.3 \pm 0.7}$%	126 ± 12
RLS (d=2)	5087 ± 1505	0.37 ± 0.19	23.2 ± 22.9%	119 ± 10
Round-Robin	6632 ± 37	0.50 ± 0.00	—	102 ± 3

Setup (G1).

Three DIO configurations: `nlms_hybrid_on` (metrics scrape + hybrid cost terms), `nlms_hybrid_off` (NLMS only, scrape disabled), and Round-Robin. All use rank-only admission (no SLO rejects).

Results.

Table 5 and Figure 4: hybrid-on mean p99 is 2000 ± 1031 ms (median 1633 ms) vs. hybrid-off 3378 ± 43 ms and RR 3388 ± 21 ms. Per-seed p99 improvement vs. RR is $\mathbf{40.9}\% \pm \mathbf{30.5}\%$ (positive on **8/10** seeds; two seeds slightly negative when load spreads). Hybrid-off is near RR on p99 under concurrent multi-turn load—scrape-enabled hybrid terms separate the arms. Stickiness under hybrid-on is 0.998 ± 0.007 (RR 0.75). High hybrid-on variance is real: warm sticky seeds keep low tails; balanced seeds look RR-like. The stable claim is that hybrid metrics improve multi-turn tails on most seeds, not that every seed matches the old n=5 point estimate.

Table 5 Real dual-T4 multi-turn hybrid suite (n=10; 12 sessions × 4 turns; live `/metrics`; concurrent load). Mean±std.

Mode	p99 (ms)	Stickiness	Aff. hit	p99 vs RR
NLMS + hybrid metrics	2000 ± 1031	0.998 ± 0.007	0.925 ± 0.018	$\mathbf{40.9 \pm 30.5}$%
NLMS, hybrid off	3378 ± 43	0.956 ± 0.012	0.855 ± 0.025	∼0%
Round-Robin	3388 ± 21	0.750 ± 0.00	0.00	—

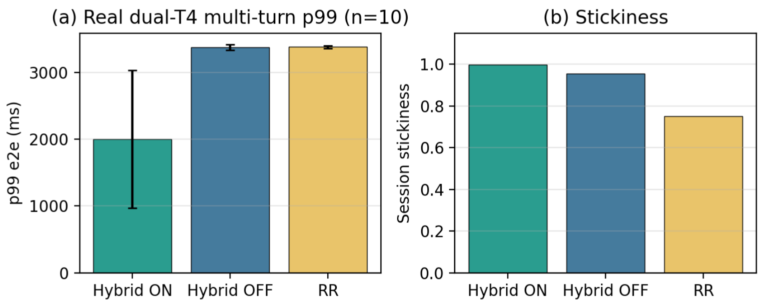


Fig. 4 Real dual-T4 multi-turn hybrid suite (n=10). (a) p99 e2e; (b) session stickiness.

7.8 Session affinity on real dual-T4 multi-turn

Setup (G2 / G2b).

Same dual-T4 engines; multi-turn with long system prefixes. G2 uses concurrent= 1; G2b uses concurrent= 2 under load. NLMS with raised cache bonus (C=400 ms) and hybrid scrape vs. Round-Robin (n=10).

Results.

Table 6 and Figure 5: under concurrent= 1, NLMS+affinity stickiness is **1.00** vs. RR **0.50**; affinity hit rate is $\mathbf{0.912 \pm 0.013}$. Mean p99 is 2335 ± 852 ms vs. RR 3386 ± 8 ms (high variance; several seeds near 1.6–1.8 s). Under concurrent load (G2b), stickiness remains $\mathbf{0.998 \pm 0.007}$ vs. RR 0.75, and hit rate is $\mathbf{0.932 \pm 0.005}$—affinity does not collapse under load. For multi-turn and agent traffic [19, 20, 23], dual-T4 evidence supports keeping turns on the warm replica without multi-SKU hardware, complementary to in-engine RadixAttention [4].

Table 6 Real dual-T4 multi-turn affinity (n=10; 12×4 turns). Mean±std. G2b = concurrent load.

Mode	Conc.	Aff. hit	Stickiness	p99 (ms)
NLMS+affinity	1	$\mathbf{0.912 \pm 0.013}$	$\mathbf{1.00 \pm 0.00}$	2335 ± 852
Round-Robin	1	0.00	0.50 ± 0.00	3386 ± 8
NLMS+affinity	2	$\mathbf{0.932 \pm 0.005}$	$\mathbf{0.998 \pm 0.007}$	2536 ± 936
Round-Robin	2	0.00	0.75 ± 0.00	3378 ± 16

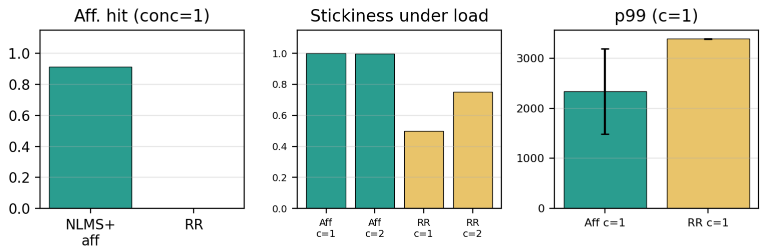


Fig. 5 Real dual-T4 multi-turn affinity (n=10). Hit rate, stickiness under load, and p99 (concurrent= 1).

7.9 Admission safety on the real gateway

Setup (G3).

Same dual-T4 pair; sequential short chat probes (40/seed, n=10). Tight SLO L=800 ms (below typical e2e on this setup) exercises admission under unreliable absolute $\hat{y}$ (Section 7.4). Modes: `absolute`, `empirical`, `rank_only`.

Results.
Table 7 and Figure 6: `absolute` rejects **39.5**% ± **21.0**% of requests (HTTP 503). `empirical` and `rank_only` reject **0**% and complete all 40 requests per seed. Completed-request p99 is similar across modes (∼3.4 s); the systems point is reject *rate*: absolute thrash under high MAPE, empirical and rank_only do not [24, 25, 46].

Table 7 Real dual-T4 admission suite (n=10 × 40; SLO L=800 ms). Mean±std. p99 is over completed requests only.

Mode	Reject rate	Completed / 40	p99 completed (ms)
`absolute`	**0.40 ± 0.21**	24.2	3415 ± 63
`empirical`	0.00 ± 0.00	40	3407 ± 43
`rank_only`	0.00 ± 0.00	40	3409 ± 57

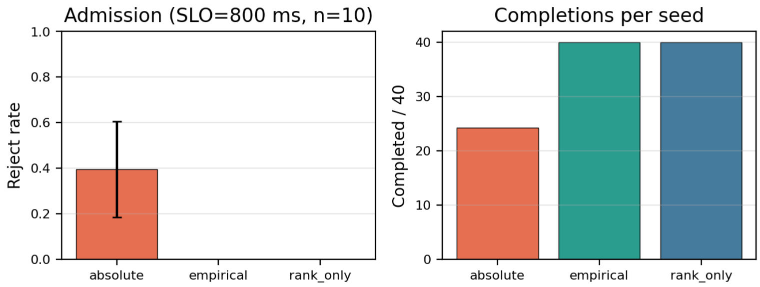


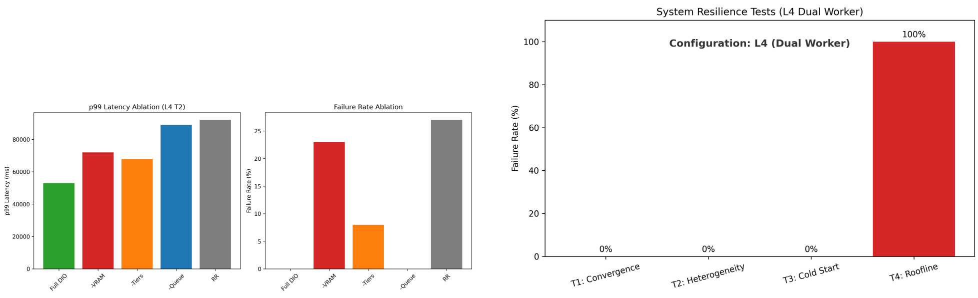
Fig. 6 Real dual-T4 admission under tight SLO (n=10). Absolute rejects ∼40%; empirical and `rank_only` reject none.

7.10 Ablations and resilience

On an L4 stressed long-prompt microbench, removing VRAM admission elevates hard failures (∼23%); removing queue wait raises tail latency; removing tiers hurts mixed-capability mixes (Figure 7a). Cold-start and overload behaviors are summarized in the same figure (b): under intentional overload, DIO returns HTTP 503 rather than unbounded queueing, which matches empirical/rank-only safety rather than absolute-$\hat{y}$ thrashing.

8 Discussion and limitations

When each mechanism matters.
NLMS ranking alone helps under observed service skew (Section 7.6), not when peers match (Section 7.3). Hybrid scrape terms matter under real multi-turn pressure with live `/metrics` (Section 7.7); without scrape, multi-turn concurrent p99 collapses toward RR. Affinity stickiness holds at concurrent= 1 and under concurrent load (Section 7.8). Prefer `empirical` or `rank_only` admission when MAPE is high (Section 7.9). In practice: enable hybrid scrape when `/metrics` exists, keep affinity on for chat/agent traffic, and do not gate production traffic on absolute $\hat{y}$ alone.



(a) Cost-term ablation (b) Resilience / overload

Fig. 7 L4 microbench ablations. (a) Cost-term ablation. (b) Resilience including overload admission (HTTP 503).

Hybrid coefficient sensitivity.

The dual-T4 hybrid headline uses defaults $(c_{\mathrm{kv}}, c_q, c_p) = (800, 50, 150)$. A library multi-seed check (n=10) scales each coefficient independently by ±50% under injected engine pressure. Under that strong asymmetry, all variants still send all traffic to the cool worker (frac cool = 1.0) and p99 is unchanged relative to default—i.e., the *routing decision* is invariant once the hybrid signal is strong enough. That does *not* replace a dual-T4 grid search of the multi-turn headline (mean ∼41% vs. RR at n=10): the magnitude of that real-engine gain may still depend on the defaults. We treat dual-T4 coefficient sensitivity as an open question and ship config overrides plus a GPU sweep mode (`run_gpu_abc_suite.py --run-coeff`).

Limitations and open strengthenings.

(i) Hybrid telemetry is limited to stock vLLM Prometheus exports (no continuous NVML SM/power loop). (ii) Regime C skew is a delay-proxy on identical T4s, not multi-SKU hardware or organic thermal/co-tenant traces—the intro's motivating organic divergence is measured only via that reproducible proxy. (iii) Title-mechanism dual-T4 multi-turn cells now use n=10, but hybrid-on and affinity p99 remain high-variance (some seeds near RR); stickiness and admission reject rates are the more stable signals. (iv) Hybrid coefficients are heuristic-default plus library ±50% invariance check, not dual-T4 grid-searched (`--run-coeff` is available). (v) Baselines are RR/LL/RLS on the same vLLM pair, not multi-node Orca/vLLM-distributed; we do not bake off every metrics-weighted Envoy/Ray Serve plugin.

9 Conclusion

DIO is a thin OpenAI-compatible gateway for stock multi-instance vLLM deployments. It ranks with dual-timescale NLMS, corrects scores with scraped engine metrics, keeps multi-turn sessions sticky when that helps, and admits traffic without trusting absolute $\hat{y}$ when MAPE is high. On dual-T4, matched short probes give no free lunch; a ×2 slow peer yields a large p99 gain; and real multi-turn runs show hybrid scrape, session affinity, and safe admission each matter in the ways the title claims. Source code and artifacts are public.

Acknowledgements. The authors thank colleagues for feedback on earlier drafts.

Declarations

- **Funding.** No external funding was received for this work.

- **Conflict of interest/Competing interests.** The authors declare that they have no competing interests.
- **Ethics approval and consent to participate.** Not applicable (no human subjects or personal data).
- **Consent for publication.** Not applicable.
- **Data availability.** Aggregated results are under `results_workshop_final/`, `results_gpu_abc/`, and related directories. Scripts regenerate library and dual-T4 suites.
- **Materials availability.** Not applicable.
- **Code availability.** Source code for `dio-serve` (hybrid `/metrics` scrape, affinity, admission modes), validation suites (`run_workshop_final_suite.py`, `run_abc_experiments.py`, delay-proxy harness), and paper artifacts are available at https://github.com/nisaral/DIO.
- **Author contribution.** K.N. led system design, implementation, experiments, and manuscript drafting. K.P. and K.M. contributed to evaluation, analysis, and manuscript revision. All authors approved the final manuscript.

Appendix A Notation summary

Table A1 Main symbols.

Symbol	Meaning
$s^{\text{fast}}, s^{\text{slow}}, s^{\text{eff}}$	Dual-timescale slopes (ms/token)
b_w	Intercept (ms)
$N, y, \hat{y}$	Tokens; actual / predicted e2e latency (ms)
S_w	Joint routing cost for worker w
Q_w, B	Pending depth; batch amortization
$\text{KV}_w, W_w, \text{Hit}_w$	Scraped KV usage, waiting count, prefix hit rate
c_{kv}, c_q, c_p	Hybrid cost coefficients (ms scale)
C	Prefix affinity bonus (ms)
L	Latency SLO (ms)

Appendix B Default coefficients

Default production coefficients: μ_{fast}=0.1, μ_{slow}=0.01, μ_b=0.005, α=0.8, B=8, c_{kv}=800, c_q=50, c_p=150, C=200, VRAM soft/hard 4096/2400 MB, metrics interval 1 s, empirical percentile p=95, recent window 64. Hybrid triple selection is heuristic parity with base terms (Section 4); not dual-T4 grid-searched. All are overridable via `DIO_*` settings.

References

[1] Kwon, W., *et al.*: Efficient memory management for large language model serving with PagedAttention. In: SOSP (2023)

[2] Zheng, L., et al.: SGLang: Efficient Execution of Structured Language Model Programs (2023)

[3] Hugging Face: Text Generation Inference. https://github.com/huggingface/text-generation-inference (2023)

[4] Zheng, L., *et al.*: SGLang: Efficient execution of structured language model programs. In: NeurIPS (2024). RadixAttention prefix cache

[5] F5, Inc.: NGINX HTTP Load Balancer. https://nginx.org/ (2024)

[6] Cloud Native Computing Foundation: Kubernetes. https://kubernetes.io/ (2024)

[7] Envoy Project Authors: Envoy Proxy. https://www.envoyproxy.io/ (2024)

[8] Ray Project: Ray Serve. https://docs.ray.io/en/latest/serve/ (2024)

[9] Yu, G.-I., *et al.*: Orca: A distributed serving system for Transformer-based generative models. In: OSDI (2022)

[10] Agrawal, A., *et al.*: Sarathi-Serve: Efficiently serving large language models via hybrid scheduling. In: OSDI (2024)

[11] Eisenbud, D.E., *et al.*: Maglev: A fast and reliable software network load balancer. In: NSDI (2016)

[12] Dean, J., Barroso, L.A.: The tail at scale. Communications of the ACM **56**(2) (2013)

[13] Zhang, Y., Chen, Y., Mo, X., Xi, A., Li, J., Wu, W.: A Predictive and Synergistic Two-Layer Scheduling Framework for LLM Serving (NexusSched) (2025)

[14] Wu, B., et al.: Llumnix: Dynamic Scheduling for Large Language Model Serving (2024)

[15] Wu, B., Liu, S., Zhong, Y., Sun, P., Liu, X., Jin, X.: LoongServe: Efficiently serving long-context large language models with elastic sequence parallelism. In: SOSP (2024)

[16] OpenAI: OpenAI API Reference. https://platform.openai.com/docs/api-reference (2024)

[17] Haykin, S.: Adaptive Filter Theory, 4th edn. Prentice Hall, Upper Saddle River,

NJ (2002)

[18] Prometheus Authors: Prometheus: Monitoring System and Time Series Database. https://prometheus.io/ (2024)

[19] Wu, Q., et al.: AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation (2023)

[20] LangChain: LangGraph: Build Resilient Language Agents as Graphs. https://github.com/langchain-ai/langgraph (2024)

[21] Gim, I., et al.: Prompt Cache: Modular Attention Reuse for Low-Latency Inference (2023)

[22] Juravsky, J., et al.: Hydragen: High-Throughput LLM Inference with Shared Prefixes (2024)

[23] Srivatsa, V., et al.: Preble: Efficient Distributed Prompt Scheduling for LLM Serving (2024)

[24] Gujarati, A., *et al.*: Serving DNNs like clockwork: Performance predictability from the bottom up. In: OSDI (2020)

[25] Zhong, Y., *et al.*: DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In: OSDI (2024)

[26] Yang, A., et al.: Qwen2.5 Technical Report (2024)

[27] Prabhu, R., et al.: vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention (2024)

[28] Wu, B., Zhong, Y., Zhang, Z., Liu, S., Liu, F., Sun, Y., *et al.*: Fast distributed inference serving for large language models. In: arXiv (2023). arXiv:2305.05920

[29] Patel, P., et al.: Splitwise: Efficient Generative LLM Inference Using Phase Splitting. arXiv preprint (2024)

[30] et al.: Mooncake: Trading between Memory and Storage for Efficient LLM Serving. arXiv preprint (2024)

[31] et al.: MemServe: Context Caching for Disaggregated LLM Serving. arXiv preprint (2024)

[32] Yao, J., et al.: CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion (2024)

[33] Cheng, Y., et al.: LMCache: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference. arXiv / open-source cache layer (2024)